INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial

Unidad de aprendizaje: Análisis y Diseño de Algoritmos

Grupo: 3BV1

Práctica 2

Algoritmos de ordenamiento: tiempo y memoria

Alumna: Molina Zavaleta Angie Lalaine

Fecha: 10 de noviembre de 2026

Bubble sort · Mergesort · Tree sort · Heapsort · Counting sort
Implementados en C++ y Python

1. Objetivo y metodología

Implementar cinco algoritmos de ordenamiento en dos lenguajes (C++ y Python), medir su tiempo de ejecución y su memoria sobre arreglos de números reales de 10^5 a 10^9 elementos, y reportar los resultados con tablas y gráficas. Las implementaciones son propias (no se usa std::sort ni sorted()).

Entorno de pruebas: Intel Xeon 2.1 GHz con **1 vCPU y 3.9 GB de RAM** (sin swap), Linux 6.18; g++ 13.3 con `-O2 -std=c++17`; Python 3.12.3 (intérprete estándar, sin NumPy).

Datos: números reales con dos decimales en [0, 100000.00), generados como k/100 con k uniforme en [0, 10^7) (semilla 42). Se eligió esta representación porque Counting sort solo es aplicable a reales cuando tienen precisión y rango acotados: la clave entera es k = round(100·x). Con reales arbitrarios de doble precisión no hay un conjunto de claves enteras acotado y el algoritmo no es práctico. Los generadores de C++ y Python son distintos (mt19937_64 y Mersenne Twister de Python), por lo que los arreglos no son idénticos, solo de la misma distribución.

Medición: cada combinación (algoritmo, n) se ejecuta en un proceso independiente. El **tiempo** es el de la rutina de ordenamiento (reloj monotónico, sin generación de datos ni verificación). La **memoria** es el pico de memoria residente (RSS) del proceso; la memoria auxiliar es el pico después de ordenar menos el pico con solo el arreglo de entrada. Cada resultado se verificó (arreglo ordenado y misma suma de claves que la entrada). Se hizo **una sola ejecución** por configuración, así que hay ruido de medición.

2. Algoritmos implementados

Algoritmo	Tiempo (mejor / promedio / peor)	Memoria auxiliar	Notas de la implementación
Bubble sort	O(n) / O(n²) / O(n²)	O(1)	Con bandera de intercambio y reducción del rango en cada pasada
Mergesort	O(n log n) en todos los casos	O(n)	Descendente, con un único buffer auxiliar de n elementos
Tree sort	O(n log n) / O(n log n) / O(n²)	O(n)	Árbol binario de búsqueda sin balancear, un nodo por elemento (16 B en C++); recorrido inorden iterativo. El peor caso ocurre con datos ya ordenados
Heapsort	O(n log n) en todos los casos	O(1)	In situ, con sift-down
Counting sort	O(n + K), K = 10^7	O(n + K)	Versión estable con sumas prefijas y arreglo de salida

3. Resultados

En las tablas, los valores normales son **medidos***; los que llevan ≈ (en cursiva) son* **estimaciones** *por escalamiento desde la mayor medición disponible (cuadrático para Bubble sort, n log n para los demás, lineal para Counting sort). Las estimaciones son orientativas y suelen ser optimistas cuando el arreglo deja de caber en caché o en RAM. La memoria ≈ de 10^9 en C++ es el requerimiento teórico. — significa no ejecutado ni estimado.*

3.1 C++

Tiempo de ejecución

Algoritmo	n = 100,000	n = 1,000,000	n = 10,000,000	n = 100,000,000	n = 1,000,000,000
Bubble sort	28.75 s	*≈ 47.9 min*	*≈ 3.3 días*	*≈ 332.8 días*	*≈ 33277.3 días*
Mergesort	9 ms	110 ms	1.28 s	15.02 s	*≈ 2.8 min*
Tree sort	19 ms	526 ms	14.21 s	5.7 min	*≈ 63.8 min*
Heapsort	11 ms	169 ms	2.67 s	45.37 s	*≈ 8.5 min*
Counting sort	32 ms	65 ms	412 ms	4.70 s	*≈ 47.02 s*

Memoria pico (RSS)

Algoritmo	n = 100,000	n = 1,000,000	n = 10,000,000	n = 100,000,000	n = 1,000,000,000
Bubble sort	3.8 MB	—	—	—	≈ 7.45 GB
Mergesort	4.7 MB	18.4 MB	155.7 MB	1.49 GB	≈ 14.90 GB
Tree sort	5.4 MB	26.0 MB	232.0 MB	2.24 GB	≈ 22.35 GB
Heapsort	3.9 MB	10.7 MB	79.4 MB	766.1 MB	≈ 7.45 GB
Counting sort	42.9 MB	56.5 MB	193.9 MB	1.53 GB	≈ 14.94 GB

3.2 Python

Tiempo de ejecución

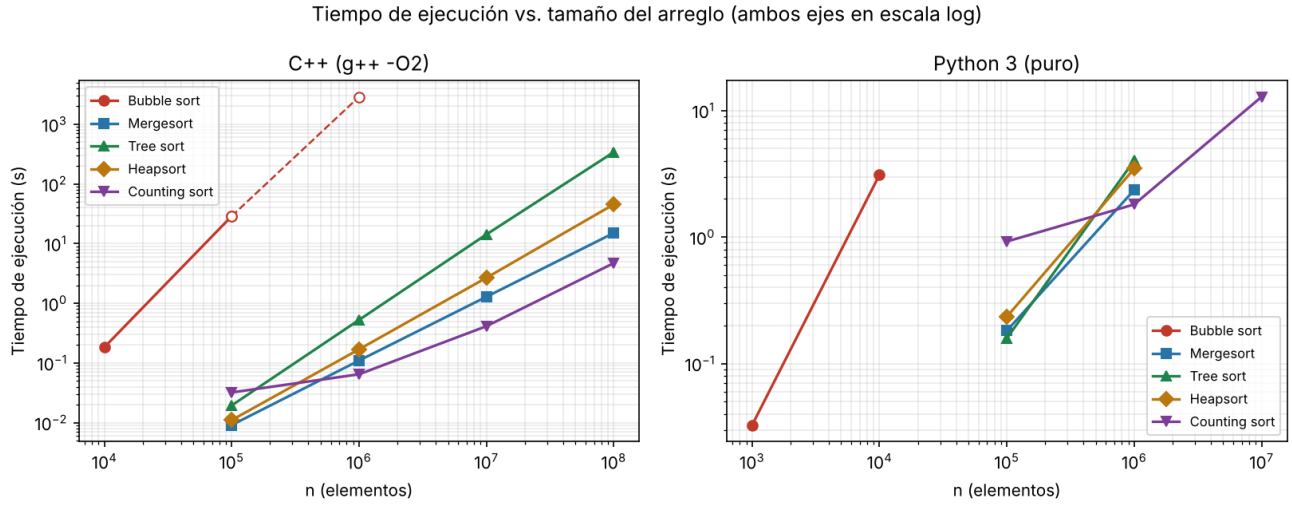
Algoritmo	n = 100,000	n = 1,000,000	n = 10,000,000	n = 100,000,000	n = 1,000,000,000
Bubble sort	*≈ 5.2 min*	*≈ 8.6 h*	*≈ 36.0 días*	*≈ 3596.9 días*	*≈ 359687.5 días*
Mergesort	182 ms	2.36 s	*≈ 27.48 s*	*≈ 5.2 min*	*≈ 58.9 min*
Tree sort	159 ms	4.03 s	*≈ 47.04 s*	*≈ 9.0 min*	*≈ 100.8 min*
Heapsort	235 ms	3.47 s	*≈ 40.53 s*	*≈ 7.7 min*	*≈ 86.9 min*
Counting sort	923 ms	1.81 s	12.84 s	*≈ 2.1 min*	*≈ 21.4 min*

Memoria pico (RSS)

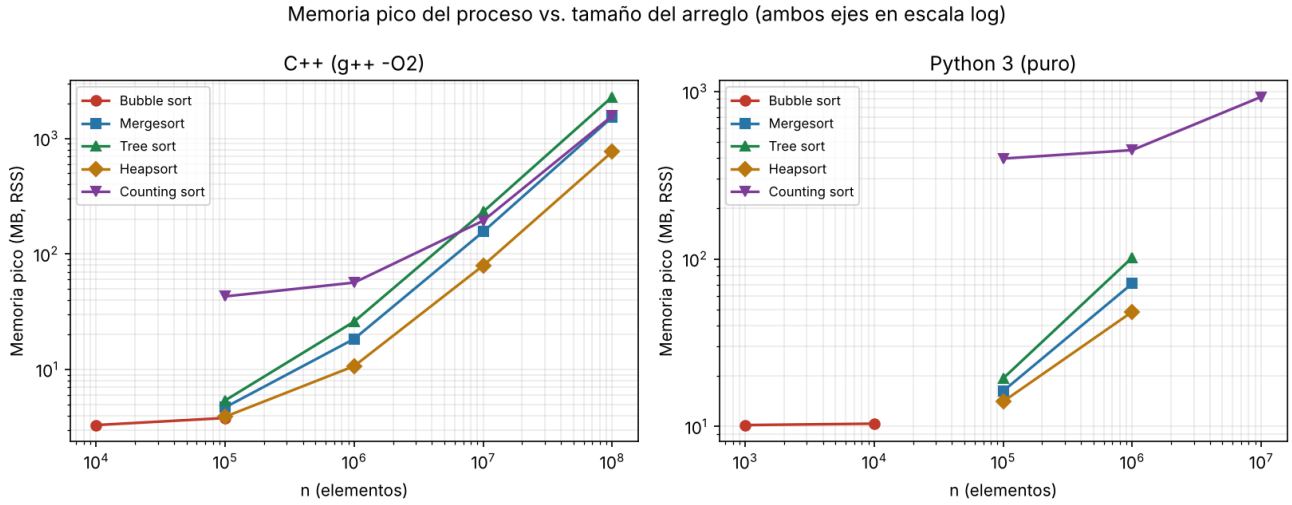
Algoritmo	n = 100,000	n = 1,000,000	n = 10,000,000	n = 100,000,000	n = 1,000,000,000
Bubble sort	—	—	—	—	—
Mergesort	16.3 MB	71.4 MB	—	—	—
Tree sort	19.4 MB	102.0 MB	—	—	—
Heapsort	14.1 MB	48.4 MB	—	—	—
Counting sort	397.4 MB	446.3 MB	928.2 MB	—	—

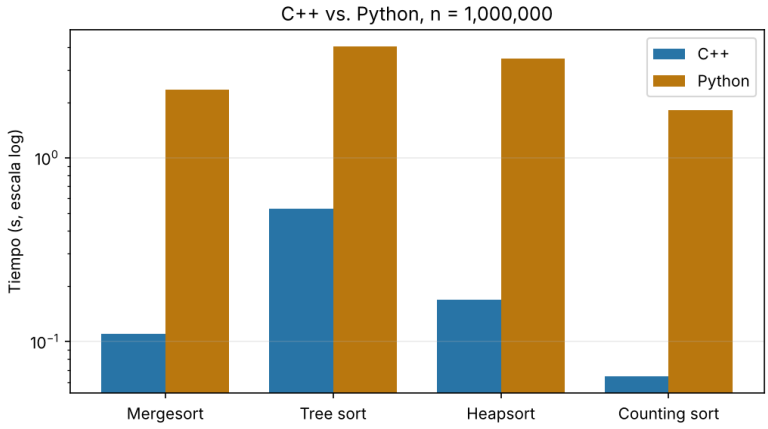
Mediciones adicionales para estimar Bubble sort: C++ $n = 10^4$: 0.183 s; Python $n = 10^3$: 0.033 s y $n = 10^4$: 3.11 s.

3.3 Gráficas



Línea discontinua con marcadores huecos: extrapolación cuadrática de Bubble sort en C++ (estimación, no medida).





4. Análisis

Bubble sort. En C++ pasó de 0.183 s ($n = 10^4$) a 28.8 s ($n = 10^5$), un factor de ×157 para un aumento de 10× en n; el crecimiento esperado es ×100 y el exceso se debe a efectos de caché y

predicción de saltos. Extrapolando, n = 10^6 tomaría al menos ≈ 48 minutos y n = 10^7 ≈ 3.3 días, por lo que no es viable para los tamaños de la práctica. En Python, n = 10^4 ya tarda 3.1 s.

Algoritmos O(n log n) en C++. Mergesort creció ≈ ×12 por cada 10× en n (0.0091 → 0.110 → 1.28 → 15.0 s), consistente con n log n. A 10^8, Mergesort (15.0 s) fue ×3.0 más rápido que Heapsort (45.4 s), porque Heapsort salta por posiciones lejanas del arreglo (muchos fallos de caché), pero necesitó la mitad de memoria (766 MB frente a 1529 MB). Tree sort fue el más lento y el que más memoria usó: 340 s y 2.24 GB a 10^8 (×23 el tiempo de Mergesort), pues recorre punteros dispersos en memoria y guarda un nodo de 16 bytes por elemento.

Counting sort. Fue el más rápido a partir de 10^6 (4.7 s a 10^8), con crecimiento casi lineal ×11.4 entre 10^7 y 10^8. Para n pequeño pierde ventaja: a 10^5 tarda 32 ms (más que Mergesort) porque inicializar y recorrer el arreglo de 10 millones de contadores domina. Su costo en memoria es 2n + K: 1567 MB a 10^8.

C++ frente a Python (n = 10^6). Python fue ≈ ×22 más lento en Mergesort, ×21 en Heapsort, ×8 en Tree sort y ×28 en Counting sort. Tree sort es el que menos pierde porque su costo ya está dominado por accesos a memoria. Además, cada float de Python es un objeto (≈ 32 bytes por elemento frente a 8): con 10^6 elementos el proceso de Mergesort llegó a 71 MB frente a 18 MB en C++. Counting sort en Python usó 397 MB incluso con n = 10^5, por la lista de 10^7 contadores enteros.

5. Limitaciones: qué no se ejecutó y por qué

• **n = 1,000,000,000 no se ejecutó en ningún algoritmo ni lenguaje.** Solo el arreglo de entrada ocupa 8 GB (doubles) y el entorno de prueba tenía 3.9 GB de RAM. Los requerimientos teóricos en C++ son ≈ 7.5 GB (Heapsort, Bubble sort), ≈ 15 GB (Mergesort, Counting sort) y ≈ 22 GB (Tree sort). Los tiempos de esa columna son estimaciones por escalamiento, no mediciones. Para ejecutarlo, usa una máquina con ≥ 32 GB de RAM y descomenta la línea correspondiente de `run_bench.sh` (solo Heapsort está previsto).

• **Bubble sort** solo se midió hasta n = 10^5 en C++ y n = 10^4 en Python; los demás tamaños son extrapolaciones cuadráticas (viable ejecutar n = 10^6 en C++, unos 50 minutos o más).

• **Python** se midió hasta n = 10^6 (y Counting sort hasta 10^7) por tiempo y memoria; los demás valores son estimaciones. Con n = 10^8, una lista de floats necesitaría ≈ 3.2 GB solo para el arreglo.

• Una sola ejecución por configuración, en un equipo virtual de 1 vCPU: los tiempos pueden variar entre ejecuciones y entre máquinas. La memoria RSS incluye el intérprete y las bibliotecas (≈ 3 MB en C++ y ≈ 10 MB en Python).

• Tree sort se evaluó solo con datos aleatorios; con datos ya ordenados su árbol se degenera en una lista y el costo pasa a $O(n^2)$ (no medido). Counting sort depende de que los reales tengan 2 decimales y rango acotado.

6. Conclusiones

Los resultados confirman las complejidades teóricas: Bubble sort es inviable más allá de 10^5 elementos; Mergesort, Heapsort y Tree sort escalan como n log n pero con constantes muy distintas por su comportamiento de caché y su memoria auxiliar (Mergesort el más rápido de los tres pero con el doble de memoria que Heapsort; Tree sort el más lento y voraz); y Counting sort es el mejor cuando el rango de claves es pequeño frente a n, a costa de memoria proporcional a n + K y de exigir claves enteras acotadas. El lenguaje importa tanto como el algoritmo: en este entorno Python fue entre 8 y 28 veces más lento que C++ y usó mucha más memoria por elemento. La escala de 10^9 elementos exige hardware con decenas de GB de RAM o algoritmos externos (por ejemplo, mergesort externo).